



SDEV2171

Lesson 04: React Native Core Components

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Identify and describe common React Native core components
- Use core components to structure a basic mobile application screen
- Explain how React Native components differ from web-based HTML elements
- Modify an existing Expo application to include additional core components
- Apply basic debugging techniques when working with UI components

Lesson Roadmap

1. Reconnect lesson-03 project to visible UI
2. Introduce core components and web analogs
3. Walk through the target screen structure
4. Guided build in the existing app
5. UI experimentation and troubleshooting
6. Checkpoint and lesson-05 handoff

Starting Point From Lesson 03

- use the same Expo project created in lesson 03
- open `App.js`
- confirm the app still runs before making larger UI changes

If your project is not ready

- recover one working path first, or
- use the instructor reference app for practice while preserving a next step for your own project

Canonical Starting State

Before lesson 04 changes, the shared student baseline is:

- one working Expo app from lesson 03
- `SafeAreaView`, `View`, and `Text` already present
- a known-good `App.js` that still runs

If your file looks different

- use [starting-state-reference.md](#) after class, or
- use the instructor recovery app during class and record which changes you made yourself

What Changed Since Lesson 03

- lesson 03 proved that one file change can affect the running app
- lesson 04 uses that same file to build a more deliberate screen
- today the key question is not “Does it run?” but “Which component should I use here?”

Core Components For Today

Component	Main job	Rough web analog
<code>View</code>	group and structure UI	non-scrolling <code>div</code>
<code>Text</code>	show readable text	<code>p</code> or <code>span</code>
<code>Image</code>	display an image	<code>img</code>
<code>ScrollView</code>	allow content to scroll	scrollable <code>div</code>
<code>TextInput</code>	collect typed input	<code>input</code> or <code>textarea</code>

Important caution

Web analogs help you orient yourself, but React Native components are not HTML tags.



ESSENTIAL
TO ALBERTA



View And Text

View

- container for grouping other components
- used for sections, cards, rows, and screen structure

Text

- every visible string belongs inside Text
- text styling and nesting behave differently from the web

Image, ScrollView, **And** TextInput

Image

- needs a valid source
- usually needs explicit width and height

ScrollView

- useful when content may extend beyond one screen
- acceptable for this lesson's small screen
- not the right tool for very long lists

TextInput

- collects user input
- gives students one simple interactive element on the screen

Target Screen Structure

```
SafeAreaView
  ScrollView
    View (screen container)
      Text (eyebrow)
      Text (title)
      Text (intro)
      View (feature card)
        Image
        Text
        Text
      View (notes section)
        Text
        TextInput
```

Why this structure

It shows containers, nesting, scrolling, media, and input in one manageable screen.

Guided Build Order

1. keep the lesson-03 baseline running
2. add `ScrollView`
3. build one new content card with `View` and `Text`
4. add `Image`
5. add `TextInput`
6. test and adjust one small layout detail

Minimal Syntax For Today

```
import { Image, ScrollView, TextInput } from 'react-native';

<ScrollView>
  <Image source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }} style={{ width: 64, height: 64 }} />
  <TextInput placeholder="Type here" />
</ScrollView>
```

Course decision

The lesson uses one shared remote image URL so the whole class can use the same Image example.

HTML Comparison Without Overgeneralizing

- `View` is like a structural container, but not a web document element
- `Text` is required for visible strings in a way the web does not require
- `Image` uses React Native props and sizing rules, not HTML attributes
- `ScrollView` is a component decision, not just a CSS overflow choice
- `TextInput` behaves like a mobile input control, not a browser form field

Intermediate Checkpoints

- after `ScrollView` : the app still runs, even if the screen looks almost the same
- after the new `View` and `Text` section: one extra section is visible
- after `Image` : one visible image block appears
- after `TextInput` : one field accepts typing

If a checkpoint fails

Stop there and debug that step before adding the next component.

Common Debugging Points

- component imported incorrectly or not imported at all
- text placed directly inside `View` without `Text`
- `Image` missing width or height
- saved file does not update the running app
- layout looks wrong because one container is nested in the wrong place

Response pattern

Check the smallest failing piece before changing multiple things at once.

Debugging Mini-Lab

Try one controlled test during class:

- temporarily remove the `Image` width or height and observe what happens
- restore the size and confirm the image returns

Why this matters

It gives you one real debugging loop instead of only reading about troubleshooting.

Guided Experimentation

- change one text block and confirm it still renders
- move a section into a different `View` and observe the structure change
- adjust image size and note the visual result
- add or remove enough text to make scrolling visible
- type into the `TextInput` and confirm the field responds

Checkpoint

Students should be ready to show

- a running screen that uses multiple core components
- one example of nested `View` and `Text` structure
- one explanation of how a chosen React Native component differs from a simple HTML assumption

Students should also be able to answer

- Which component did you add first?
- Why did you choose `ScrollView` or keep it?
- What troubleshooting step helped or would help if the UI broke?

What We Learned Today

- which React Native core components appear most often in basic screens
- how to use `View`, `Text`, `Image`, `ScrollView`, and `TextInput` for a small screen
- why React Native components are similar to, but not identical to, HTML elements
- how component nesting affects screen structure
- how to debug common UI component mistakes step by step

Next Lesson Bridge

- keep this same project and `App.js` file
- lesson 05 focuses on styling the structure you built today
- next class improves spacing, alignment, readability, and visual clarity

Ready for lesson 05 means

- the app still runs
- the screen still uses the core components you added
- you can identify which parts of the screen need clearer spacing or styling
- some visual issues still remain on purpose, such as plain spacing, rough alignment, or basic text hierarchy